Arreglos inteligentes con C++: <vector>



V5.0

FERREIRAS

MARZO 2010

¿ QUE ES UN ARREGLO ?

- Es una forma de almacenar información en bloques de celdas contiguas de RAM como una colección ordenada en secuencia de elementos de datos de un mismo tipo bajo un mismo nombre.

DIRECCION	INDICE	INFORMACION
...		
FFF101	0	f
FFF102	1	G
FFF103	2	@
...	...	...
FFF500	499	7
FFF501	500	$
...		

DIRECCION EN RAM	FFF01	FFF02	FFF03	...
ARREGLO A_i DE ENTEROS	A_1	A_2	A_3	...
VALOR ALMACENADO	12	1234	-7893	...

Si hacemos: $A_1 = A_1 + A_2$; $A_2 = -456$, Entonces, los nuevos valores almacenados en cada A_i son:

DIRECCION EN RAM	FFF01	FFF02	FFF03	**...**
ARREGLO A_i DE ENTEROS	A_1	A_2	A_3	**...**
VALOR ALMACENADO	1246	-456	-7893	**...**

- Cada elemento es identificado con una variable subindicada;
- Las variables subindicadas se comportan como las variables simples, esto es, podemos hacer las mismas operaciones (asignación, ...);
- Cada elemento almacena el mismo tipo de dato;

- Cada elemento, en diferentes momentos, puede ir almacenando diferentes valores, del mismo tipo;

- El arreglo entero es almacenado en bloques contiguos de RAM. Esto es, no hay huecos entre un elemento y el siguiente;

- Cada elemento es identificado por un subíndice único, esto es, un elemento se distingue de otro por medio del subíndice;
- En un arreglo dado puede haber elementos con valores almacenados iguales pero nunca con el mismo subíndice;
- Los subíndice se numeran desde 0, esto es, 0, 1, 2, 3 ...

EJEMPLOS DE ARREGLOS

- Un arreglo de butacas: B_o, B_1, B_2, B_3, ..., B_i

  Cada B_i solo "almacena" un estudiante, aunque puede "almacenar", en diferentes momentos, diferentes estudiantes;

				...
B0	B1	B2	B3	...

EJEMPLOS DE ARREGLOS

- Una imagen puede ser representada como un gran arreglo de lecturas de brillo, un elemento para cada una de las miles de celdas en que la imagen es dividida;

- Un arreglo de pixeles;

TIPOS DE ARREGLOS EN C++

⇨ De tamaño fijo en tiempo de compilación:

T ID[n];

n es el número de elementos en el arreglo;

⇨ **De tamaño fijo en tiempo de ejecución**, usando punteros y los operadores de C++ ***new*** y ***delete***:

```
T *pID = new T[n]; // n entero

...

delete *pID;
```

En el tema de punteros veremos esto;

⇨ **De tamaño variable en tiempo de ejecución:** Son los arreglos inteligentes o `<vector>`, que desarrollaremos en este curso;

⇨ **<array>**

⇨ **<valarray>**

LOS CUALES ESTÁN FUERA DEL ALCANCE DE ESTE CURSO.



ARREGLOS DE TAMAÑO FIJO EN TIEMPO DE COMPILACIÓN

T ID[n];

Declara ID un arreglo vacío de n elementos tipo T

T ID[n] = { v_1, v_2, v_3, ..., v_n} ;

Declara ID un arreglo de n elementos y cada elemento es inicializado con un valor v_i de tipo T

```
/*
  array0.cpp
  Compilador: bcc32 5.5

  C++ / Ferreiras / Arreglos inteligentes: <vector>

  Arreglos estaticos de tamano fijo.-

*/

#include <iostream>
 using namespace std;
```

```
int main(void){
   system("cls");
   const unsigned int tamano(5);
   double a[tamano];// Declara un array de size fijo tamano
   int indice(0);
   cout << "\n\n => Entre numeros reales, CTRL+Z termina\n" << endl;
   while( cin >> a[indice] ){
     ++indice;
     if( indice == tamano)
       break;
   }
   cout << "=>\n Ultimo valor del indice: " << indice << endl;
   double suma(0);
   for (int i(0); i < indice; i++) {
        suma = suma + a[i]; // Acumula los valores a[i]
   }
   cout << "\nLa suma de los reales suplidos es: " << suma << endl;
   return 0;
}
```

Ver array0.cpp

```
 => Entre numeros reales, CTRL+Z termina

12.34
23.45
34.56
45.67
56.78

=> Ultimo valor del indice: 5

La suma de los reales suplidos es: 172.8

=> Entre numeros reales, CTRL+Z termina

1.23
2.34
^X
=> Ultimo valor del indice: 2

La suma de los reales suplidos es: 3.57
```

Observar que el último valor del índice siempre será igual a la cantidad de elementos suplidos. Esto permite su uso en el for.

ARREGLOS DE CADENAS DE CARACTERES

⇨ `char cadena[80];` define una cadena como un arreglo de caracteres; Es una **cadena al estilo C**; Para manipular este tipo de cadena se requiere de la librería <cstring>; *Permite trabajar con una cadena de **tamaño fijo** desde el momento de su declaración.*

```
#include <iostream>
#include <cstring>                                   // cstr0.cpp

using namespace std;

int main(void){

  char cad1[ ]="Esto es una cadena al estilo C";
  char cad2[40];
  char cad3[40];

 // USO DE LA FUNCION strcpy( c2, c1)

  strcpy(cad2, cad1);
  strcpy(cad3, "Copiando a cad3");

  cout << endl << cad1 << endl
          << endl << cad2 << endl
          << endl << cad3 << endl;
  return 0;
}
```

```
/*

C:\Borland\BCC55\Bin>cstr0

Esto es una cadena al estilo C

Esto es una cadena al estilo C

Copiando a cad3

*/
```

<vector>



¿ QUE ES UN <vector> C++ ?

- **Es el arreglo oficial de C++;**
- Esta implementado en la librería <vector>;
- Es un arreglo que puede crecer dinámicamente, esto es, ***su tamaño es variable en tiempo de ejecución;***
- Si T es un tipo, entonces, `vector<T> v1` define v1 como un arreglo dinámico de elementos de tipo T;

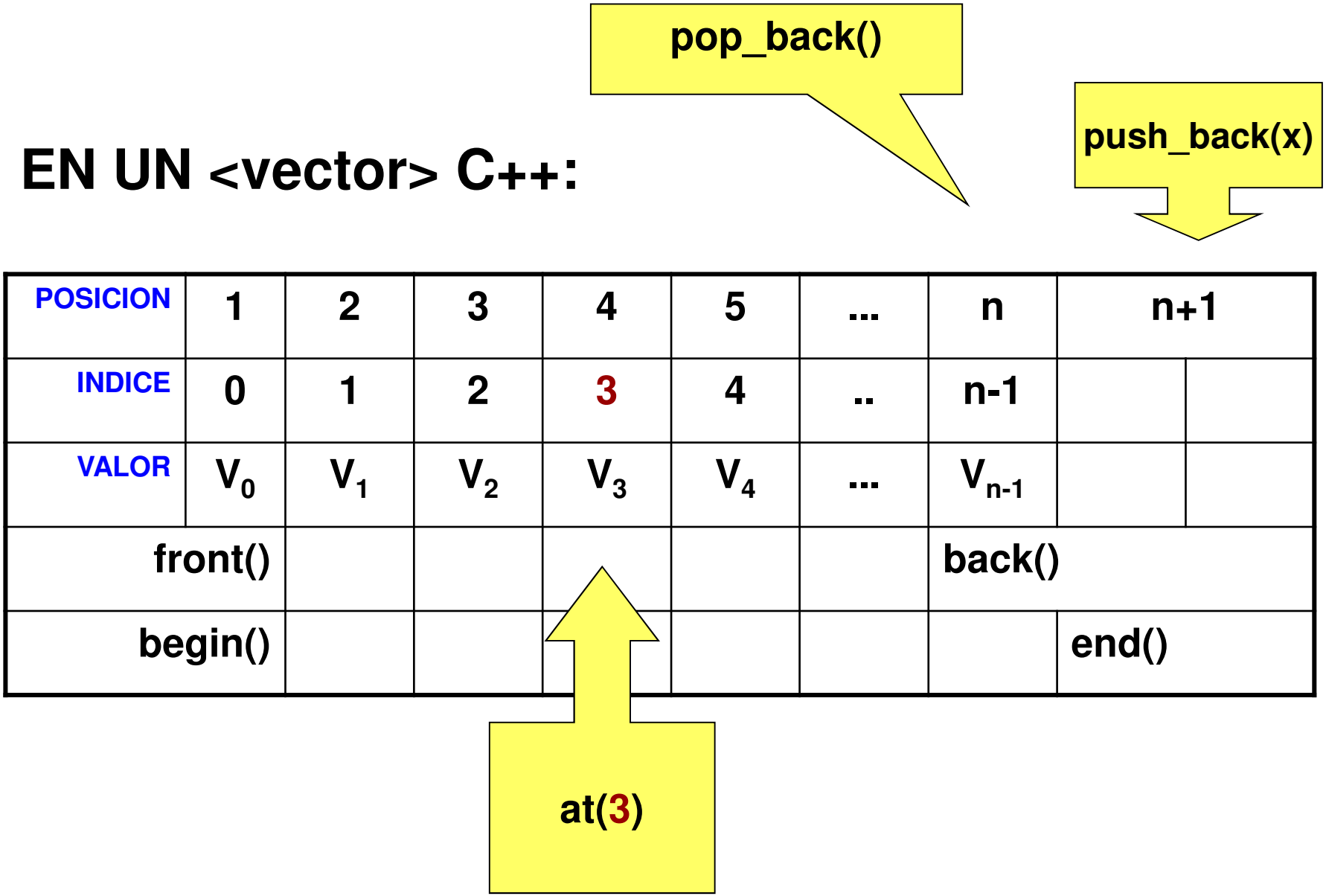
- Es un arreglo inteligente, a diferencia de los arreglos regulares, el almacenamiento es manejado de manera automática, permitiendo ser expandido o contraído a requerimiento en tiempo de ejecución;

- Son más eficientes y seguros accesando a sus elementos por el subíndice;
- Permiten acceso aleatorio de sus elementos en cualquier orden;

- Permiten su redimensionamiento con facilidad;

- Permiten añadir y remover elementos después del y en el último;

EN UN <vector> C++:



DECLARACION (DEFINICION) DE UN <vector> EN C++

- Requiere `#include <vector>`
- `vector<T> id;` // Declara un <vector> de tipo T, vacio
- `vector<T> id(Ke);` /* Declara un <vector> de tipo T, con una cantidad inicial de Ke elementos */
- `vector<T> id(Ke, KVi);` /* Declara un <vector> de tipo T con Ke elementos y KVi valores iniciales */

•• **EJEMPLOS (segmento de código)**

```
vector<float> tasa;
vector<int> est41101(25);
vector<string> nombres(12, "Freddy
                           Pérez");
```

ver vect0.cpp

RECURSOS EN <vector> de C++

vectorID.recurso

· assign	end	operator=	reserve
at	erase	operator[ ]	resize
· back	front	pop_back	size
· begin	get_allocator	push_back	swap
· capacity	insert	rbegin	
· clear	max_size	rend	
empty			

- **`capacity(), max_size(), size()`**

  - **`capacity()`** Es la capacidad de almacenamiento actualmente asignada, expresada como cantidad de elementos;

  - **`max_size()`** Máximo número de elementos que el <vector> puede almacenar;

  - **`size()`** Cantidad de elementos en el <vector>

- **`VER vect11.cpp`**

- **OPERACION DE ASIGNACION EN <vector>**

```
vect2 = vect1;
```

- erase() todos los elementos en vect2;
- insert() en vect2 una copia de cada uno de los elementos en vect1;
- Después de la asignación vect2 y vect1 tienen el mismo size y comparan a igual uno con otro;

VER vect10.cpp

- **AGREGANDO Y RETIRANDO ELEMENTOS A UN <VECTOR>;**

•• **`push_back`** Agrega un nuevo elemento al <vector> justamente después del último;

•• **`pop_back`** Remueve el último elemento del <vector> reduciéndolo en 1 y borrando toda referencia a dicho elemento retirado;

••Ejemplo:

`VER vect12.cpp; vect0.cpp; vect1.cpp;`

- **ACCESANDO A LOS ELEMENTOS DE UN <vector>**

  - Usando el recurso `at(indice);`
  - Usando un `iterator;`
  - Usando el operador de subíndice `[indice];` **NO ES RECOMENDADO, a(t) ES MAS SEGURO;**

Todos permiten acceder al elemento en el <vector> y hacer operaciones como lo hacemos con cualquier variable simple.

- **EJEMPLO:** `vect1.cpp; vect2.cpp`

- **EL RECURSO** `vector<T>::iterator`

  - `vector<T>::iterator id;` Declara un apuntador llamado `id` que permite accesar a los elementos en el <vector> en forma análoga al uso del indice pero más segura.

    - Identifica el <vector> y un elemento en este;

- Nos permite examinar, con `*id`, el valor almacenado en ese elemento;

- Nos provee operaciones, `*(id + entero)`, para movernos entre los elementos del <vector>;

- Restringe las operaciones a las que <vector> puede manejar con eficiencia;

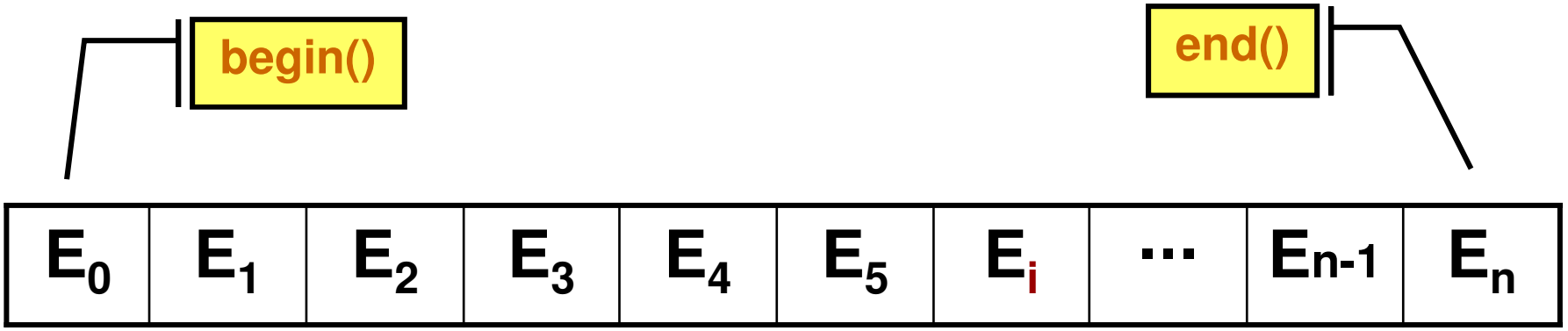
VER vect2.cpp

- **RECURSOS** `begin() y end()`

  - `begin()` retorna un apuntador al primer elemento del <vector>, esto es, `vector.at(0); o vector.front();`

  - `end()` retorna un apuntador a la posición **uno más** después del último elemento; **`En el <vector> actual at(n) no existirá`**

GRÀFICAMENTE:



EL <vector> ACTUAL SIEMPRE VA DESDE i = 0 HASTA i = n-1

VER vect2.cpp

EJEMPLOS (segmento de código)

`vector<double>::iterator iter1, iter2;`
Declara iter1, iter2 como iteradores de vectores de double;

`vector<double> v1;`

`iter1= v1.begin();` Pone el iter1 a apuntar el primer elemento del <vector> de dobles v1;

`*iter1` Accesa al valor almcenado en la posición a la que actualmente apunte;

VER vect2.cpp

- **OPERADORES DE IGUALDAD**

  == Y !=

- **OPERADORES DE RELACION CON <vector>**

  <, >, <= y >=

VER ARCHIVO `vector_comparacion.doc`

VER `vect9.cpp`

- **MAS RECURSOS EN <vector>**

  `insert(), erase(), assign(), clear(),`

  `empty(), swap(), ...`

VER EL ARCHIVO `vector_recursos.doc`

`VER vect3.cpp; vect4.cpp; vect6.cpp; vect7.cpp; vect8.cpp;`

clear()

⇨ **El recurso clear()** de <vector> remueve todos los elementos en un <vector> dado, dejándolo con un size de 0 y liberando la memoria asignada.

```
vector<T> v;

...

v.clear();
```

⇨ **POR EJEMPLO**:

=> La capacidad del <vector> actual es: 0

=> La capacidad del <vector> actual es: 64

=> El contenido actual en el <vector> es:

Estamos en <vector>, ! que bien !
Y tenemos 6 puntos
Para el next lab

=> Estoy vacio, mi size es: 0 elementos

=> La capacidad del <vector> actual es: 64

=> La capacidad del <vector> actual es: 64

=> Me han empujado elementos y ahora tengo 2, los cuales son:

Ahora le cargo un elemento
Otro elemento

Ver el código completo en vectclear0a.cpp

⇨ Según el comité estándar de C++, clear() no cambia la capacidad del <vector>, únicamente cambia su size;

⇨ En el ejemplo **`vectclear0a.cpp`** se ilustra lo indicado anteriormente.

erase()

⇨ **El recurso erase ()** de <vector> remueve del vector un elemento de una posición indicada o un rango de elementos [primero, último].

```
vector<T>v;
…
v.erase(pos);
v.erase(pos1, pos2);
```

⇨ **`v.erase(pos);`** Remueve el elemento en la posición `pos;`

⇨ **`v.erase(pos1, pos2 );`** Remueve los elementos en el rango [pos1, pos2];

⇨ **`pos, pos1 y pos2`** deben ser iteradores del <vector>

⇨ **POR EJEMPLO**

=> El contenido del <vector> v es:

1.23 12.33 13.44 14.55 15.66 16.77 17.88

Si hacemos un v.erase (v.begin() + 5);

=> El contenido ahora es:

1.23 12.33 13.44 14.55 15.66 17.88

Si hacemos v.erase(v.begin() + 2, v.begin() + 4);

=> El contenido ahora es:

1.23 12.33 15.66 17.88

⇨ Ver código completo en vecterase1.cpp

MATRICES

UNA MATRIZ ES UN ARREGLO DE DOS DIMENSIONES:

e_{11}	e_{12}	…	e_{1c}
e_{21}	e_{22}	…	e_{2c}
…	…	…	…
e_{f1}	e_{f2}	…	e_{fc}

f : Es fila

c: Es columna

La declaración de una matriz es:

- Estática (tamaño fijo en tiempo de compilación):

T m[f][c];

- Dinámica (tamaño fijado en tiempo de ejecución:

Usando **new** y **delete** $\Rightarrow$ A ver en el tema de punteros.

- **Como un tipo de dato Matrix**

  Esta implementado usando técnicas de templates a ver en el siguiente nivel INF-512;

- **Como un <vector> de <vector>**

  A ver mas adelante en esta PPT como <vector> multidimensional.

⇨ Ejemplo de matriz estática.

matr0.cpp
Compilador: bcc32 5.5

C++ / Ferreiras / Matrices / estatica

La empresa SEGUROS 4V&1F. S.R.L. tiene 15 vendedores de seguros y almacena manualmente sus ventas mensuales en una tabla como esta:

e11	e12	e13	e14	e15
e21	e22	e23	e24	e25
e31	e32	e33	e34	e35

⇨ Ejemplo de matriz estática.

La empresa es propiedad de Don Crispolio el cual para cuadrar su tabla suma las ventas en cada fila, las ventas en cada columnas, las sumas de las ventas en cada fila, las sumas de las ventas en cada columna y verifica que estas dos ultimas sumas sen iguales. Si son iguales la tabla esta cuadrada y si no la tabla no esta cuadrada.

El siguiente codigo C++ implementa esta metodo manual.

```
#include <iostream>
 using namespace std;

int main(void){

   const unsigned int fila(3), col(5);
   double tabla[fila][col];
   unsigned int f, c;

   // CARGA DE LA MATRIZ CON DATOS

   cout << "\n\n=> CARGA DE LA MATRIZ, ENTRE VENTAS:\n\n";
   for( f = 0; f < fila; ++f){
      cout << "fila # " << f << endl;
      for( c = 0; c < col; ++c ){
          cout <<"   columna # " << c << ": ";
          cin >> tabla[f][c];
      }
   }
```

```
// PROCESAMOS LOS DATOS ACCEDIENDO A CADA ELEMENTO DE LA
// MATRIZ

   // SUMA DE LAS VENTAS EN CADA COLUMNA c EN UNA FILA f

  double totalfila[fila];
  for( f = 0; f < fila; ++f)
    for( c = 0; c < col; ++c )
       totalfila[f] = totalfila[f] + tabla[f][c];

 // SUMA DE LAS VENTAS EN CADA FILA f EN UNA COLUMNA c

  double totalcol[col];
  for( c = 0; c < col; ++c)
    for( f = 0; f < fila; ++f )
       totalcol[c] = totalcol[c] + tabla[f][c];
```

```
// SUMAMOS LOS TOTALES

    double sumatotalfilas(0);
    for( f = 0; f < fila; ++f)
      sumatotalfilas = sumatotalfilas + totalfila[f];

    double sumatotalcol(0);
    for( c = 0; c < col; ++c)
       sumatotalcol = sumatotalcol + totalcol[c];
```

```
// VERIFICAMOS LAS SUMAS

    int verif(sumatotalfilas - sumatotalcol); // para redondear
    if( verif == 0 )
      cout << "\n\n => La tabla esta cuadrada" << endl;
    else
      cout << "\n\n => La tabla no esta cuadrada" << endl;

    cout << "\n=>    Suma de totales por filas: " << sumatotalfilas
         << "\n\n=> Suma de totales por columnas: "
         << sumatotalcol
         << "\n\n=> Verificacion: " << verif << endl;

    return 0;

}
```

```
C:\Borland\BCC55\Bin>matr0


=> CARGA DE LA MATRIZ, ENTRE VENTAS:

fila # 0
   columna # 0: 1.11
   columna # 1: 2.33
   columna # 2: 4.55
   columna # 3: 0.000000056
   columna # 4: 3.44
fila # 1
   columna # 0: 5.66
   columna # 1: 6.77
   columna # 2: 7.88
   columna # 3: 8.88
   columna # 4: 3.33
fila # 2
   columna # 0: 2.22
   columna # 1: 11.1
   columna # 2: 22.2
   columna # 3: 3.33
   columna # 4: 4.44
```

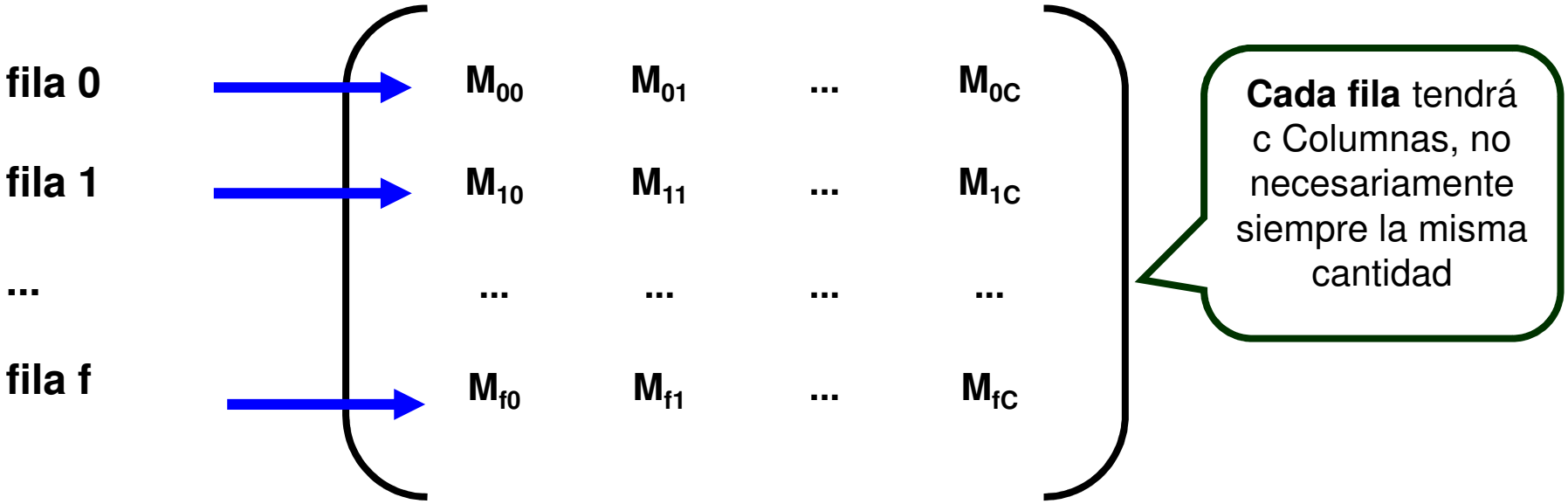
=> La tabla esta cuadrada

=> Suma de totales por filas: 87.24

=> Suma de totales por columnas: 87.24

=> Verificación: 0

<vector> MULTIDIMENSIONALES



vector< vector<T> > M

El <vector> interior es el que define las filas

⇨ **M es un <vector> de f filas** cada una de las cuales es, también, un **<vector> de c Columnas**; M no es necesariamente rectangular, esto es, en cada fila, c podrá estar en el rango [0, m], $\forall$ m $\in$ E^+

⇨ **Para procesar cada elemento** de la matriz debemos primero acceder a la fila y luego al elemento de esa fila, con cualquiera de los siguientes métodos, propios de los <vector>:

- M.at(f).at(c);
- M.at(f)[c];
- M[f].at(c);
- M[f][c]

Ejemplo

vect2d0.cpp
Compilador: bcc32 5.5

C++/ Ferreiras/ <vector> / 2D

Para trabajar con matrices:

(1) usar la librería Matrix en www.boost.org

(2) Usar un vector de vectores

```
vector< vector<T> > id_matr; // deje los espacios indicados
```

(3) Usar

```
typedef vector<T> columnas;
vector<columnas> filas; // Cada fila tiene c Columnas
```

(4) Tanto en 2 como en 3, después se deberán crear las filas inyectándole vectores con push_back, previamente cargados. Cada vez que inyectamos un vector es una fila nueva en la matriz. Por ejemplo:

vector < vector<int> > matr;// Crea una matriz vacía
matr.push_back(vector <int>()); // inyecta una fila vacía
matr[f].push_back(*valor*); // Inyecta *valor* en la fila matr[f]

Luego Podemos acceder a los elementos de la matriz usando la notación de subíndices matr[f][c], donde f y c son valores de subíndices validos.

En el siguiente código ilustramos esto mas ampliamente.

!! Cada fila de la matriz podrá tener la cantidad de elementos que el usuario quiera !!

```
#include <vector>
#include <iostream>
 using namespace std;

int main(void){

  // DECLARACION DE LA MATRIZ

    vector< vector<int> > matr; // crea una matriz vacia

    // Inyectamos la fila 0, matr[0]
    matr.push_back( vector <int>() ); // inyecta una fila vacia
    // Inyectamos valores en la fila matr[0]
    matr[0].push_back(50); //Inyecta 50 a matr[0] en matr[0].at(0)
    matr[0].push_back(33); //Inyecta 33 a matr[0] en matr[0].at(1)
    matr[0].push_back(12); //Inyecta 12 a matr[0] en matr[0].at(2)
    matr.at(0).push_back(987); // Inyecta 987 en matr.at(0).at(3)
```

```
// Mostrar contenido en fila 0
cout << endl << "matr.at(0).at(0)= " << matr.at(0).at(0);
cout << endl << "  matr.at(0)[1] = " << matr.at(0)[1];
cout << endl << "   matr[0].at(2)= " << matr[0].at(2);
cout << endl << "      matr[0][3]= " << matr[0][3];

// Inyectamos la fila 1, matr[1]
matr.push_back( vector <int>() ); // inyecta otra fila vacia
// Inyectamos valores en la fila matr[1]
matr[1].push_back(111); //Inyecta 111 a matr[1] en matr[1].at(0)
matr[1].push_back(633); //Inyecta 633 a matr[1] en matr[1].at(1)
matr[1].push_back(765); //Inyecta 765 a matr[1] en matr[1].at(2)
matr.at(1).push_back(1023); // Inyecta 1023 en matr.at(1).at(3)
```

```
// Mostrar contenido en fila 1
    cout << endl;
    cout << endl << "matr.at(1).at(0)= " << matr.at(1).at(0);
    cout << endl << "  matr.at(1)[1] = " << matr.at(1)[1];
    cout << endl << "   matr[1].at(2)= " << matr[1].at(2);
    cout << endl << "      matr[1][3]= " << matr[1][3];
   // ...
    cout << endl;
    return 0;
}
```

```
/*

C:\Borland\BCC55\Bin>vect2d0

matr.at(0).at(0) = 50
   matr.at(0)[1]  = 33
    matr[0].at(2) = 12
        matr[0][3] = 987

matr.at(1).at(0) = 111
   matr.at(1)[1]   = 633
    matr[1].at(2)  = 765
        matr[1][3]  = 1023

*/
```

vect2d1b.cpp

```
#include <vector>
#include <iostream>

 using namespace std;

int main(void){

  // DECLARACION DE LA MATRIZ

    vector< vector<int>  > matr;

 // CARGA DE LA MATRIZ
 // CADA VEZ QUE INYECTEMOS UN VECTOR A LA MATRIZ, ES UNA FILA
NUEVA

    vector<int> x; // Para las filas
    int y;
    char resp;
```

Ese espacio es obligatorio

```
do {
      // CARGAMOS LOS ELEMENTOS DE LA FILA A SER INYECTADA

      cout << "\n=> Entre los elementos de la fila a inyectar:"
             << " CTRL+Z para terminar\n" << endl;
      x.clear(); // borra los elementos en el vector, si los hay.
       // Si el vector no es clear(), simplemente se van agregando
       // dinámicamente y los elementos de las filas anterior
       // estarán en la fila siguiente. (\_~)

      while(cin >> y)
        x.push_back(y); // carga los elementos de la fila
```

```
// INYECTAMOS LA FILA A LA MATRIZ

      matr.push_back(x); //inyectamos el vector o fila nueva
      cin.clear(); // limpia el buffer

     // REQUERIMOS OTRA FILA  PARA LA MATRIZ

      cout << "\nOTRA FILA ? s/n: ";
      cin >> resp;

 } while(resp == 's' || resp == 'S');
```

```
// PROCESAMOS LOS ELEMENTOS DE LA MATRIZ, POR
// EJEMPLO, MOSTRARLOS

   cout << "\n\nCantidad de filas = " << matr.size() << endl;
   for(unsigned f(0); f < matr.size(); f++) {
        cout << "\nCantidad de columnas en la fila " << f
                << " es " << matr.at(f).size() << endl
                << "Estos elementos son: " << endl;
        for (unsigned j(0); j < matr[f].size(); j++) {
             cout << matr[ f ] [ j ] << " ";
     }
        cout << endl;
   }

   return 0;
}
```

```
C:\Borland\BCC55\Bin>vect2d1b

=> Entre los elementos de la fila a inyectar: CTRL+Z para terminar
1
2
3
4
5
^Z
^Z

OTRA FILA ? s/n: s
```

```
OTRA FILA ? s/n: s

=> Entre los elementos de la fila a inyectar: CTRL+Z para terminar
6
7
8
9
10
11
12
13
14
15
^Z
^Z
```

```
OTRA FILA ? s/n: s

=> Entre los elementos de la fila a inyectar: CTRL+Z para terminar

16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
^Z
^Z
```

OTRA FILA ? s/n: **n**

Cantidad de filas = 3

Cantidad de columnas en la fila 0 es 5
Estos elementos son:
1 2 3 4 5

Cantidad de columnas en la fila 1 es 10
Estos elementos son:
6 7 8 9 10 11 12 13 14 15

Cantidad de columnas en la fila 2 es 16
Estos elementos son:
16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

C:\Borland\BCC55\Bin>

Obsérvese que no necesariamente tenemos que trabajar con una matriz rectangular

```
/*
  vect2d0a.cpp
  Compilador: bcc32 5.5

  C++/ Ferreiras/ <vector> / 2D

  => Usando la definición de tipos <vector>:

            typedef   vector<T> columnas;
            typedef   vector<columnas> filas;

*/

#include <vector>
#include <iostream>
 using namespace std;
```

```
int main(void){

    // c Es cantidad de columnas
    // f  Es cantidad de filas

   typedef vector<int> v1; // vo, v1, ... , v(c-1)
   typedef vector<v1>  v2; // <vector> de vectores v1

   v1 fila; // fila0, fila1, fila2, ..., fila(c-1)
   v2 m; //   m0, m1, ... , m(f-1)

   // INYECCION DE FILAS Y COLUMNAS A LA MATRIZ

   m.push_back(fila); // inyecta una fila vacia
   m[0].push_back(50); //Inyecta 50 a m[0] en m[0].at(0)
   m[0].push_back(33); //Inyecta 33 a m[0] en m[0].at(1)
   m[0].push_back(12); //Inyecta 12 a m[0] en m[0].at(2)
   cout << endl << "m[0][0]= " << m[0].at(0); // muestra m[0][0]
   cout << endl << "m[0][1]= " << m[0][1]; // muestra m[0][1]
   cout << endl << "m[0][2]= " << m[0].at(2); // muestra m[0][2]
```

```
    m.push_back( fila ); // inyecta otra fila vacia
    m[1].push_back(51); //Inyecta 51 a m[1] en m[1].at(0)
    m[1].push_back(78); //Inyecta 78 a m[1] en m[1].at(1)
    m[1].push_back(432); //Inyecta 432 a filas[1] en m[1].at(2)
    cout << endl << "m[1][0]= " << m[1][0]; // muestra m[1][0]
    cout << endl << "m[1][1]= " << m[1][0]; // muestra m[1][1]
    cout << endl << "m[1][2]= " << m[1].at(2); // muestra m[1][2]

    cout << endl;
    return 0;
}
```

```
/*

C:\Borland\BCC55\Bin>vect2d0a

m[0][0]= 50
m[0][1]= 33
m[0][2]= 12
m[1][0]= 51
m[1][1]= 51
m[1][2]= 432

C:\Borland\BCC55\Bin>

*/
```

<valarray>

C++ provides the standard library **<valarray>**, which is designed to carry out vector operations very efficiently.

By now this topic is out of scope of this course. I am so sorry.

Ferreiras

<array>

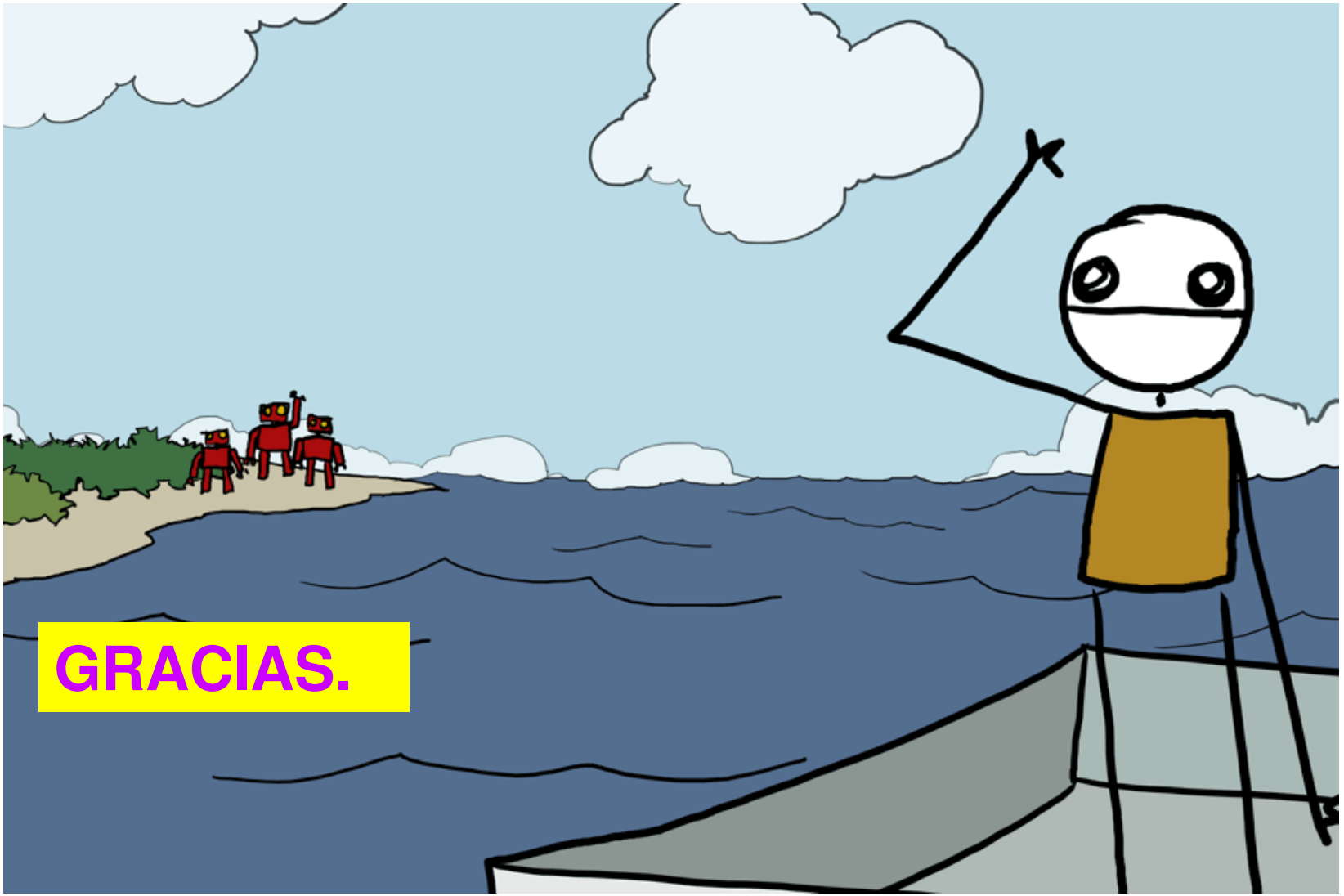
⇨ **<array>** Es la nueva herramienta de C++, en su versión **C++09**, como una alternativa eficaz a <vector> ***para la manipulación de arreglos de tamaño fijo.*** Obviamente, solo con aquellos compiladores que sean C++09 es que podremos usarla, bcc32 5.5 no lo es. ;-)

⇨ **Por ejemplo**, en un compilador C++09, las siguientes instrucciones usan <array>

```
array<float,3>  A = {{1., 2., 3.}};

array<float,3>  B;

array<float,3>  C = {{10., 20., 30.}};
```


GRACIAS.